

Android 13~16

Sensor Hub闭源库编译与链接指导手册

WWW.UNISOC.COM

修改历史

版本号	日期	注释
V1.2	2025/04/27	- 将文档名修改为《Android 13~16 Sensor Hub闭源库编译与链接指导手册》 - 添加Android 16相关平台的sensor hub代码路径
V1.1	2024/06/03	将文档名修改为《Android 13 ~ 15 Sensor Hub闭源库编译与链接指导手册》。
V1.0	2024/01/17	第一次正式发布： - 优化文档描述 - Sensor Hub编译环境介绍中增加 “Sensor Hub代码目录”
V0.1	2023/01/11	临时发布。

关键字

关键字：Sensor Hub、闭源库、编译、链接

目录



01 Sensor Hub编译环境介绍

02 常用CMake语句介绍

03 闭源库编译与链接示例

01

Sensor Hub 编译环境介绍

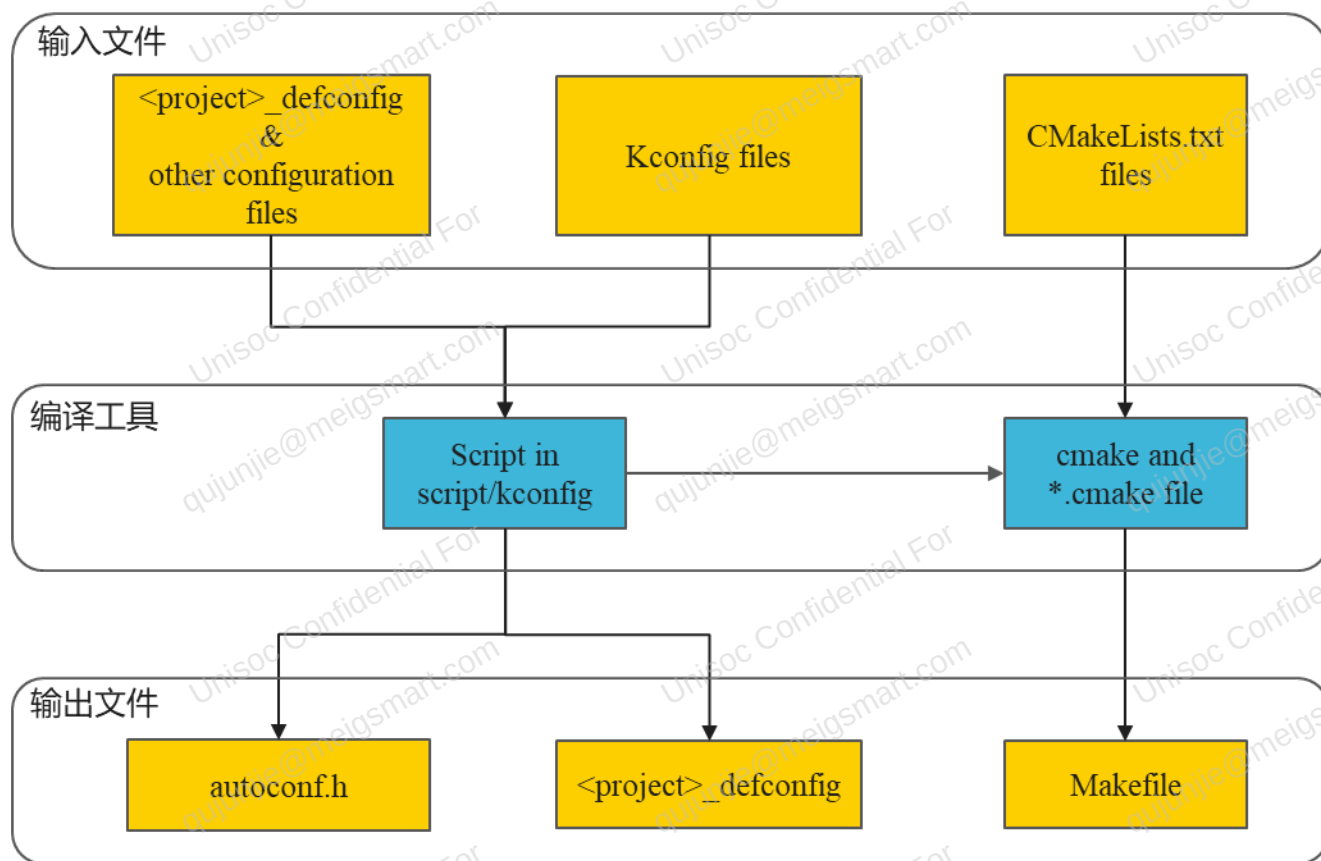
编译工具

Sensor Hub子系统使用CMake+Make(GNU)+Kconfig+GCC工具链搭建编译系统。

工具	说明	链接
CMake	组织整个编译工程，CMake是生成Makefile的上游软件。	https://cmake.org/
Make (GNU)	用CMake生成的Makefile编译整个工程。	https://www.gnu.org/software/make/
Kconfig	用于生成编译时的配置选项及配置文件，其使用及配置语法与Linux Kernel Kconfig相同。	https://www.kernel.org/doc/html/latest/kbuild/kconfig-language.html
GCC (GNU Compiler Collection, GNU编译器套件)	具体使用的编译工具链。	https://gcc.gnu.org/

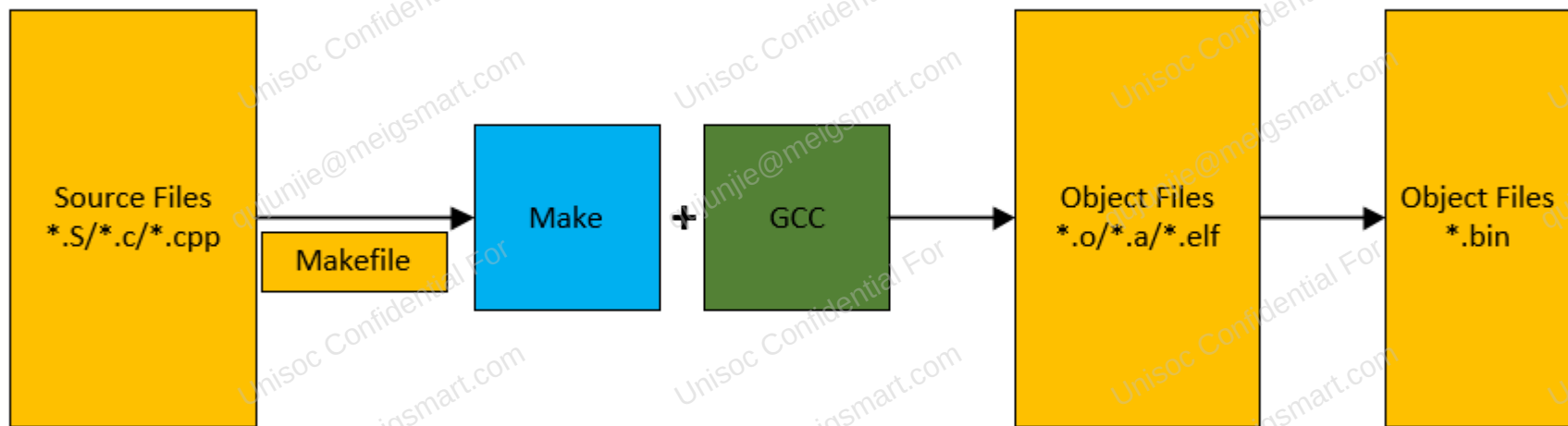
配置过程

1. Kconfig脚本利用<project>_defconfig、其他配置文件以及工程中的Kconfig文件来生成<project>_defconfig及autoconf.h。生成的文件包含工程所有功能配置，可控制代码编译。
2. CMake工具调用脚本将CMakeLists.txt和.cmake文件转换成Makefile文件。



编译过程

Make解析Makefile文件，并调用GCC对工程中的汇编、C、CPP文件进行编译，生成.o/.a/.elf文件，然后再经优化、链接生成.axf或.elf文件，最终经转换生成.bin文件。



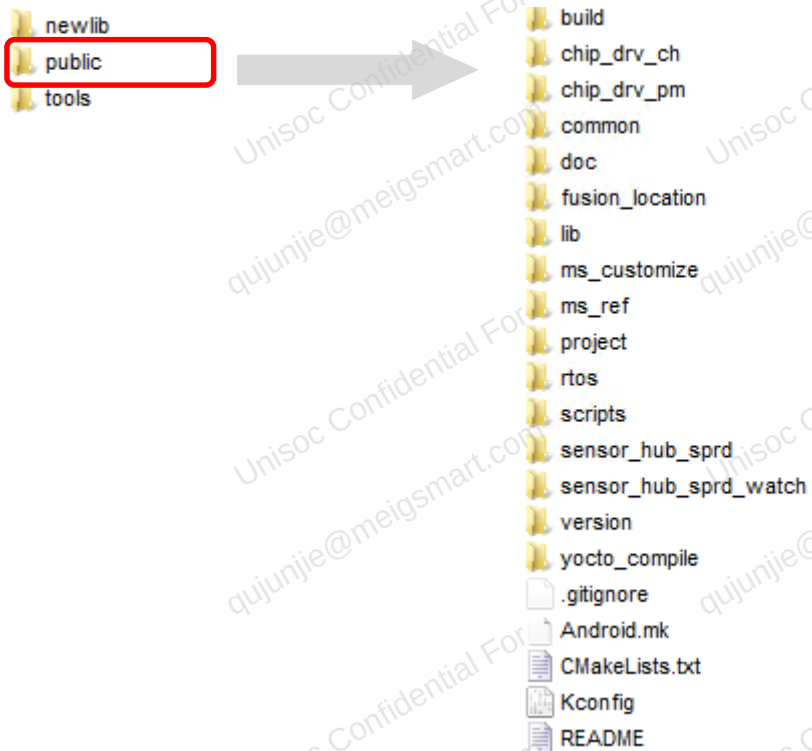
Sensor Hub代码目录 1/3

不同Android版本Sensor Hub代码路径存在一定差异：

- Android 13上所有Sensor Hub工程的代码均在bsp/sensorhub目录下。
- Android 14~16：
 - ✓ SC9863*、UMS512*、UMS312*、UMS9230*、UIS786*平台代码在bsp/sensorhub目录下
 - ✓ UMS9620*、UMS9621*、UMS9632*、UMS9360*、UIS7885*、UIS7870SC*、UIS6870W*平台代码在bsp/contexthub目录下

Sensor Hub代码目录 2/3

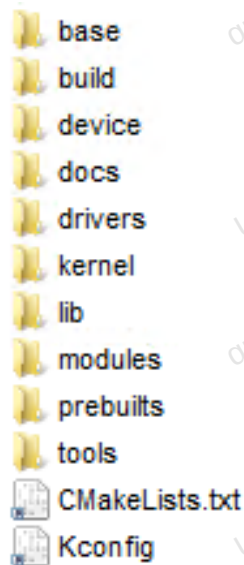
bsp/sensorhub目录如下图所示，包括Sensor Hub相关的开源代码、newlib库和相关编译配置工具。主要代码在public目录下，作为统一的仓库管理，包括CMake和工程配置模块、RTOS模块、芯片驱动模块、Sensor Hub驱动模块以及其他公共模块等。



- 如果新建模块是开源代码，可以在bsp/sensorhub/public下新建目录，与现有代码一起管理，也可以在bsp/sensorhub下新建目录，作为独立的仓库管理。
- 如果新建模块是闭源库，为了与开源代码隔离，直接在bsp/sensorhub下新建模块目录，作为独立的仓库管理。

Sensor Hub代码目录 3/3

bsp/contexthub目录如下图所示，其功能与public下功能基本一致，但是各模块新建了独立的仓库进行维护。



如果新建模块，不限制在哪个目录下新建模块目录，编译配置需要根据目录动态调整，但是如果目录下已有独立的仓库，则不能在该目录下新建仓库。对于闭源库来说，建议新建仓库与现有代码区分开，方便管理维护。

02

常用CMake语句介绍

CMakeLists.txt模板

CMakeLists.txt模板如下图所示，可基于此模板新建模块的CMakeLists.txt。本小节对CMakeLists.txt中常用的语句进行介绍。

CMakeLists.txt模板路径：

- bsp/sensorhub/public/(tools)/scripts/module_template/unisoc
- bsp/contexthub/build/cmake/module_template/unisoc

```
#####  
# 第一部分，声明所需要的cmake 工具版本  
#####  
cmake_minimum_required(VERSION 3.13)  
  
#####  
# 第二部分，编译成MMM 的库，0 表示模块开源1 表示模块闭源  
#####  
cp_internal_library_register(MMM 0)  
  
#####  
# 第三部分，描述需要编译哪些文件及使用宏控的示例  
#####  
set(SRCS  
    "src/XXX.c"  
)  
  
if(CONFIG_XXX)  
    list(APPEND SRCS  
        "src/ZZZ.c"  
    )  
endif()  
  
#####  
# 第四部分，声明编译该库时可以从哪些路径查找头文件  
#####  
cp_library_include_directories(  
    xxx/inc/  
)  
  
#####  
# 第五部分，声明该模块哪些路径下的头文件可被外部引用  
#####  
cp_include_directories(  
    xxx/inc  
)  
  
#####  
# 第六部分，编译SRCS 中有的文件，与第三部分对应  
#####  
cp_library_sources(${SRCS})
```


1. 在现有工程中新建模块

chip_drv、sensorhub等文件夹的代码都是以模块形式存在，这些目录下的代码会先被编译成.a，保存在以下文件夹内，代码链接时会把lib下的库全链接上。

- bsp/sensorhub/public/build/\$project/lib
- bsp/contexthub/out/\$project/\$board/lib

可按如下方法新建模块：

- 新建模块目录，放入代码文件，并在该目录中放入**新建模块**的CMakeLists.txt和Kconfig（如需配置宏控）文件。
- 在**上级目录**的CMakeLists.txt和Kconfig中添加新建模块。
 - ✓ 在**上级目录**的CMakeLists.txt中添加add_subdirectory(**新建模块**)。
 - ✓ 在**上级目录**的Kconfig中source “**新建模块**/Kconfig”。

示例：

以新建ms_ref模块为例，需要新建ms_ref模块目录，并在上级目录的CMakeLists.txt和Kconfig中进行以下修改。

- 在bsp/sensorhub/public/CMakeLists.txt文件中添加以下代码：

```
add_subdirectory(ms_ref)    // 添加ms_ref模块
```

- 在bsp/sensorhub/public/Kconfig文件中添加以下代码：

```
source “ms_ref/Kconfig”    // 包含ms_ref模块的Kconfig文件
```

常用CMake语句 2/4

2. 条件语句

```
if(变量)
    语句块
endif()
```

或者使用如下语句

```
if(变量)
    语句块
elseif (变量)
    语句块
endif()
```

示例:

在bsp/sensorhub/public/CMakeLists.txt文件中添加以下代码:

```
if(CONFIG_SENSORHUB_CODE_SELECT_SENSOR_HUB_SPRD_WATCH)
    add_subdirectory(sensor_hub_sprd_watch)
elseif(CONFIG_SENSORHUB_CODE_SELECT_SENSOR_HUB_SPRD)
    add_subdirectory(sensor_hub_sprd)
endif()
//如果定义了
CONFIG_SENSORHUB_CODE_SELECT_SENSOR_HUB_SPRD_WATCH, 则
添加sensor_hub_sprd_watch模块, 否则如果定义了
CONFIG_SENSORHUB_CODE_SELECT_SENSOR_HUB_SPRD, 则添加
sensor_hub_sprd模块。
```


3. 在已有模块中新增源文件

如果需要往已有模块中增加新的代码文件，修改该模块的CMakeLists.txt或.cmake文件（类Makefile文件），使新增文件能被编译到。修改方法如下：

- 赋值SRCS变量
 - ✓ 使用set (SRCS xxx)赋值SRCS变量，声明当前模块需要编译的文件。注意，当一个CMakeLists.txt文件中有多条set(SRCS xxx)时，只有最后一条生效。
 - ✓ 也可以使用list(APPEND SRCS xxx)追加文件到SRCS变量中。

以上语句中的xxx代表具体的文件路径。

- 声明需要检索的头文件路径

使用cp_library_include_directories()声明头文件路径。

示例：

在bsp/sensorhub/public/sensor_hub_sprd/public/system/sensor_driver/acc_drivers/acc_drivers.cmake文件中添加以下代码：

```
cp_library_include_directories(  
    ${SENSOR_DRIVER_PATH}) //声明${SENSOR_DRIVER_PATH} 作为头文件路径  
list(APPEND SRCS  
    "${ACC_DRIVERS_PATH}/sensor_driver_accelerometer_common.c") //追加 "sensor_driver_accelerometer_common.c" 文件到  
SRCS变量中，使其加入编译
```

常用CMake语句 4/4

4. 新增库文件

根据开发需求，在CMakeLists.txt或.cmake文件中按照如下语法添加闭源库。

- `cp_append_export_library(xxx/xxx.a)` //链接闭源静态库文件，`xxx/xxx.a`代表闭源库的路径/文件名
- `cp_setup_external_lib_include(xxx)` //声明库相关头文件路径

示例：

以链接地磁静态库为例，在`bsp/sensorhub/public/sensor_hub_sprd/public/system/sensor_driver/mag_drivers/mag_drivers.cmake`文件中添加以下代码：

```
cp_append_export_library(${MAG_DRIVERS_PATH}/akm_cali/libAK09911.a) //链接${MAG_DRIVERS_PATH}/akm_cali路  
径下名为libAK09911.a的地磁静态库
```

03

闭源库编译与链接示例

闭源库编译与链接示例 1/4

以UMS9230平台新增一个third_sensor_algo闭源库为例，展示如何在现有Sensor Hub编译环境的基础上，编译链接一个闭源库。bsp/contexthub中路径配置会有一定差异，但是总体实现方案一致，示例中不再展开区分。

1. 新增第三方算法库源文件

在bsp/sensorhub目录下新增third_sensor_algo文件夹，文件夹中主要包含以下文件：

- third_sensor_algo.c：源文件
- third_sensor_algo.h：头文件
- CMakeLists.txt：third_sensor_algo相关的编译配置文件

当前third_sensor_algo.c中仅仅写了一个test函数作为示例，可根据实际需求进行扩展。

```
void test_third_sensor_algo(void)
{
    SENSORHUB_TRACE("Hello world");
}
```

CMakeLists.txt主要信息如下，模块名为third_sensor_algo，cp_internal_library_register中参数为1表示闭源。

```
# Create Library and add Current Library to project library list
#####
#
# Create Library
cp_internal_library_register(third_sensor_algo 1)

set(SRCS
    "third_sensor_algo.c"
)

#
#####
# Public current header file to total project
# Add the directory to INTERFACE LIBRARY
#####
cp_library_include_directories(
    ${PROJECT_SOURCE_ROOT}/sensor_hub_sprd/public/system
    ./
)
```

闭源库编译与链接示例 2/4

2. 将新增模块加入编译

修改bsp/sensorhub/public/CMakeLists.txt文件，修改如下图所示。

```
.if(CONFIG_SENSOR_HUB_CODE_SELECT_SENSOR_HUB_SPRD_WATCH)
.....add_subdirectory(sensor_hub_sprd_watch)
.elseif(CONFIG_SENSOR_HUB_CODE_SELECT_SENSOR_HUB_SPRD)
.....add_subdirectory(sensor_hub_sprd)
.endif()
+if(${THIRD_SENSOR_ALGO})
+..add_subdirectory(third_sensor_algo)
+endif()
.if(NOT ${LINK_PRIVATE_LIB})
...add_subdirectory(private/rtos)
...if(CONFIG_SUBSYS_CONFIG_CH)
```

- THIRD_SENSOR_ALGO是为了控制源码是否参与编译。当有源码时，使用源码编译，当没有源码时，直接链接闭源库。此宏控由编译函数控制，[步骤4](#)中会介绍此宏控。
- 当使用源码编译时，编译完成后，bsp/sensorhub/public/lib/unisoc下面会生成或更新对应的闭源库（注意前面有lib的前缀）。

librtos_sharkl3_cm4.a	2023/12/14 16:58	A 文件	2,153 KB
librtos_sharkl5pro_cm4.a	2023/11/24 9:05	A 文件	2,153 KB
libsensorhub_algo_qogirl6_cm4.a	2023/11/2 14:59	A 文件	665 KB
libthird_sensor_algo.a	2023/12/14 21:43	A 文件	9 KB

闭源库编译与链接示例 3/4

3. 链接闭源库

修改bsp/sensorhub/lib/libsetup.cmake, 将新添加的闭源库进行链接。

```
.....cp_append_export_library(${PROJECT_SOURCE_ROOT}/lib/unisoc/libarm_drv_${PROJECT}.a)
.....cp_append_export_library(${PROJECT_SOURCE_ROOT}/lib/unisoc/librtos_${PROJECT}.a)
.....cp_append_export_library(${PROJECT_SOURCE_ROOT}/lib/unisoc/libchip_drv_private_${PROJECT}.a)
+ -> cp_append_export_library(${PROJECT_SOURCE_ROOT}/lib/unisoc/libthird_sensor_algo.a)
.
.....set(PROJECT_C_LIB_PATHS ${NEW_LIB_ROOT}/newlib-nano/arm-none-eabi/lib/thumb/v7e-m/fpv4-sp/hard)
.....set(PROJECT_C_LIB_INC ${TOOLCHAIN_PATH}/lib/gcc/arm-none-eabi/9.2.1/include)
```


4. 修改编译函数，控制源码/算法库编译

Sensor Hub进行编译时，应该满足以下策略：当源码存在时，使用源码编译链接，当源码不存在时，直接链接存在的闭源库。

修改bsp/build/envsetup.sh中make_sensorhub函数（ContextHub中要修改make_contexthub函数）以满足此策略。

```
if [[ ! -d "$BSP_ROOT_DIR/sensorhub/private/" ]]; then
-   $BSP_SENSORHUB_CMAKE/cmake -S $BSP_SENSORHUB_SOURCE -B $BSP_SENSORHUB_BIN_PATH -DPROJECT=$BSP_SENSORHUB_PROJECT -DBOARD_INFO=$BSP_SENSORHUB_BOARD \
-   -DLINK_PRIVATE_LIB=1 -DFUSION_LOCATION_SOURCE=0 -DMY_NAME="\$(whoami)" -DCOMPILE_DATE="\$(date +%Y%m%d)" -DCOMPILE_TIME="\$(date +%H%M%S | cut -b 1-8)"
+   if [[ ! -d "$BSP_ROOT_DIR/sensorhub/third_sensor_algo" ]]; then
+       $BSP_SENSORHUB_CMAKE/cmake -S $BSP_SENSORHUB_SOURCE -B $BSP_SENSORHUB_BIN_PATH -DPROJECT=$BSP_SENSORHUB_PROJECT -DBOARD_INFO=$BSP_SENSORHUB_BOARD \
+       -DLINK_PRIVATE_LIB=1 -DFUSION_LOCATION_SOURCE=0 -DTHIRD_SENSOR_ALGO=0 -DMY_NAME="\$(whoami)" -DCOMPILE_DATE="\$(date +%Y%m%d)" -DCOMPILE_TIME="\$(date +%H%M%S | cut -b 1-8)"
+   else
+       cp -r $BSP_ROOT_DIR/sensorhub/third_sensor_algo $BSP_SENSORHUB_SOURCE
+       $BSP_SENSORHUB_CMAKE/cmake -S $BSP_SENSORHUB_SOURCE -B $BSP_SENSORHUB_BIN_PATH -DPROJECT=$BSP_SENSORHUB_PROJECT -DBOARD_INFO=$BSP_SENSORHUB_BOARD \
+       -DLINK_PRIVATE_LIB=1 -DFUSION_LOCATION_SOURCE=0 -DTHIRD_SENSOR_ALGO=1 -DMY_NAME="\$(whoami)" -DCOMPILE_DATE="\$(date +%Y%m%d)" -DCOMPILE_TIME="\$(date +%H%M%S | cut -b 1-8)"
+   fi
else
    cp -r $BSP_ROOT_DIR/sensorhub/private $BSP_SENSORHUB_SOURCE
    if [[ ! -d "$BSP_ROOT_DIR/sensorhub/fusion_location" ]]; then
@@ -2176,6 +2182,7 @@
    rm -rf $BSP_SENSORHUB_SOURCE/private
    rm -rf $BSP_SENSORHUB_SOURCE/fusion_sensor_public
+   rm -rf $BSP_SENSORHUB_SOURCE/third_sensor_algo
```

- 在原有编译命令的基础上，增加了THIRD_SENSOR_ALGO参数的传递，如果没有third_sensor_algo目录，直接链接闭源库编译，THIRD_SENSOR_ALGO参数为0，如果有此目录，该参数为1。[步骤2](#)中会根据此参数决定源码是否参与编译。当存在源码时，将源码拷贝到\$BSP_SENSORHUB_SOURCE(bsp/sensorhub/public)下，编译完成后再进行删除。bsp/sensorhub/public目录作为Sensor Hub编译的根目录。
- 其余编译命令与原有相同。
- if [[! -d "\$BSP_ROOT_DIR/sensorhub/private"]]条件表示sensorhub下没有private目录（private为平台闭源的代码目录，客户端无此目录，应在此条件下修改编译命令）。

谢谢



本文件所含数据和信息都属于紫光展锐（上海）科技股份有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

紫光展锐 UNISOC、 展讯、Spreadtrum、SPRD、锐迪科、RDA及其他紫光展锐的商标均为紫光展锐（上海）科技股份有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。